Bulacan State University
College of Industrial Technology

DOCUMENTATION
Individual Web Portfolio for Industry & OJT Placement

Course Code & Title	IIDD2 – Industrial Interface Design and Development II
Project Name	Individual Web Portfolio for Industry & OJT Placement
Student Full Name	John Carlo B. Estacio
Student ID	2023101345
Program & Section	BIT-4CG1(CT)
Instructor Name	Vinna Niña O. Ramos
Active Deployment URL	https://loyyyyestacio.github.io/ojt-portfolio/
Date of Submission	November 6, 2026

Section 1: Developer Profile & Links

Target Industry Track / Role
Junior Front/Back End Developer Intern

Live Deployment URL
https://loyyyyestacio.github.io/ojt-portfolio/

Public GitHub Repository Link
https://github.com/loyyyyestacio/ojt-portfolio

Professional Elevator Pitch
I am a Bulacan State University College of Industrial Technology student who builds both the interface and the back-end logic of apps, including LearnHive, an offline capable mobile app. I test and debug carefully, as I did on an Arduino-based automatic temperature controlled fan, and I document my work clearly. I am looking for an OJT placement where I can contribute to real front-end and back-end tasks.

Cover Letter

Dear Hiring Manager,

I am writing to apply for an On-the-Job Training placement as a Junior Front/Back End Developer Intern at your company. I am a student at the Bulacan State University College of Industrial Technology, and I am available starting the 2nd Semester of A.Y. 2026–2027 to complete my 500 training hours.

My portfolio at https://loyyyyestacio.github.io/ojt-portfolio/ shows what I can bring to your team. In LearnHive, an offline-capable mobile app, I worked on both the front end and the back end, designing the screens and building the logic that saves data on the device. In my Arduino-based automatic exhaust fan controller, I served as tester and debugger, checking how the fan responded at different room temperatures and fixing the faults I found. I also build responsive layouts with HTML5, CSS3, and basic JavaScript, and I use Git and GitHub for version control.

I would welcome the chance to learn from your team and contribute to real tasks. You can reach me at loyyyyestacio@gmaill.com, and my resume can be downloaded from my portfolio. Thank you for your time and consideration.

Sincerely,
John Carlo B. Estacio

Section 2: Technology Stack & AI Tooling

1. Coding Environment & Core Stack

- **Editor:** Visual Studio Code, with the Live Server extension to preview the site while editing.
- **Core languages:** HTML5 for structure, CSS3 (Flexbox, CSS Grid, custom properties, media queries, and keyframe animation) for design, and vanilla JavaScript for functionality. The project uses no CSS framework.
- **Browser testing:** a web browser, resized to narrow and wide windows, to check the mobile and desktop layouts.
- **Version control:** Git and GitHub.
- **Fonts:** Bricolage Grotesque and Figtree from Google Fonts, with system fallbacks.

2. AI / Vibe Coding Tools Used

I used **Claude** (by Anthropic) Sonnet 5.5 to scaffold the portfolio. I attached screenshots of the requirement sheet and my photos, and described my profile and projects in plain language. I chose it because it could read the requirement screenshots directly, produce the HTML, CSS, and JavaScript as separate files that match the required folder structure, and revise them quickly whenever I sent a follow-up. The same conversation was also used to draft my resume and this documentation.

3. Deployment Platform

I chose **GitHub Pages**. It is free for public repositories, publishes straight from the repository with no build step, and suits a static site made of plain HTML, CSS, and JavaScript. My repository also serves as the public code link the requirements ask for, and any change I commit to it updates the live site within a minute or two.

Section 3: Prompt Engineering Strategies & Iteration Log

1. Prompting Strategies Applied

Strategy 1: Context-setting. My first prompt included the full requirement sheet as screenshots, plus the facts only I could supply my full name, my target role, and the details and roles for two projects (LearnHive as front-end and back-end developer, and the Arduino exhaust fan controller as tester and debugger). Giving the exact sections, folder structure, and content upfront meant the first draft already matched the required hero, about, skills, projects, contact, and footer sections.

Strategy 2: Constraint-based and component-by-component prompting. I set firm rules in the prompt: HTML for structure, an external CSS file for design and animation, JavaScript for functionality, and a layout that responds to different screen sizes. After the first build, I changed one component at a time (the profile photo shape, the photos, the GitHub link, the email address) instead of asking for a full rewrite, which kept the parts that already worked intact.

2. Prompt Iteration Log (Before and After)

Instance A: Layout / Component Issue

Initial Prompt Given: "Generate an interactive website that is responsive to different screen sizes. Use HTML for structure, external CSS for design and animation, and JavaScript for functionality," followed by my name, target role, and project details.

Flaw in AI Output: The AI framed my profile photo and the header logo as hexagons to match a honeycomb theme. The shape was a design choice I had not asked for, and I wanted a standard circular profile picture.

Corrective Follow-Up Prompt / Strategy Used: "Make the profile icon logo circle, insert in the profile the first pic I gave." The AI replaced the hexagon clipping with a square frame and a fully rounded border radius, so my photo displays as a circle.

Instance B: Styling / Behavioral Issue

Initial Prompt Given: "Make the portfolio website look clean and modern, with a dark theme."

Flaw in AI Output: The AI produced a dark theme but treated "clean and modern" as a loose suggestion. It hardcoded color values and font sizes throughout style.css and used inline styles in the HTML, so there was no consistent design system. Spacing and font sizes varied from section to section, the text contrast on the project cards was too low to read comfortably, and the layout had no responsive breakpoints, so the navigation bar and project grid overflowed on mobile screens. The AI satisfied

the visual keyword (“dark”) but ignored the usability and consistency that “clean and modern” implies.

Corrective Follow-Up Prompt / Strategy Used: I broke the vague styling request into explicit requirements: define the colors, fonts, and spacing as CSS variables in :root, remove all inline styles, keep a minimum contrast ratio for body text, and add media queries for tablet and mobile widths. The AI then restructured the stylesheet around a single set of variables, and the layout adapted correctly across screen sizes. Changing the theme later only required editing the variables in one place.

Section 4: Real Bottlenecks & Practical Problem-Solving

Bottleneck 1: Layout & Mobile View Quirks

The LearnHive screenshot is a tall phone screen (about 274 by 628 pixels), while the project card shows images in a wide landscape frame. With the default “cover” cropping, the app’s logo and Start button would have been cut off. I found this by comparing the card at wide and narrow window sizes. The fix was to show that image whole using object-fit contain on a soft gradient background that matches the app screen. I also checked the responsive rules: at 900 pixels wide or less, the two-column sections stack into one column, and at 760 pixels or less, the menu collapses into a hamburger button.

Bottleneck 2: Live Deployment / Asset Loading Issues

My photos came as WebP and PNG files, but the code pointed to JPG file names, which would show broken images online. File names also have to match exactly, including capital letters, because GitHub Pages treats Profile.jpg and profile.jpg as different files. I resolved it by converting the images to JPG with the exact names the code expects, or by changing the file extension in the src attribute when I kept a PNG. When uploading to GitHub, I learned to upload the contents of the project folder (so that index.html sits at the root) instead of the outer folder, which would have put the site one level too deep. Empty folders also need a placeholder file because Git does not keep empty folders.

Bottleneck 3: Working with AI-Generated Code

The hardest part was that the AI had to guess details it could not know, so the first draft contained placeholder values: my degree, graduation year, training hours, LinkedIn link, and project links. I searched the code for the “TODO” notes and replaced each one with real information. I also caught small data errors that would have caused real problems, such as a GitHub username with the wrong number of letters and an email domain that looks misspelled. To keep the site consistent, the AI used one set of color and font variables in the stylesheet, so a change made in one place carried across every section, and I reviewed each section in the browser before deploying.

Section 5: Experience Reflection & OJT Readiness

1. Human vs. AI Roles

The AI did best at speed and structure: it generated clean semantic HTML, a consistent CSS design, responsive breakpoints, and the JavaScript for the menu, theme toggle, and fan preview, and it produced the exact folder structure within minutes. It needed my manual intervention for everything personal or external: my real details and links, my

photos, creating the GitHub repository and turning on GitHub Pages, testing the live site, and catching mistakes such as wrong file extensions and misspelled account details. The AI produced the draft, but I was responsible for checking that every fact was true and every link worked.

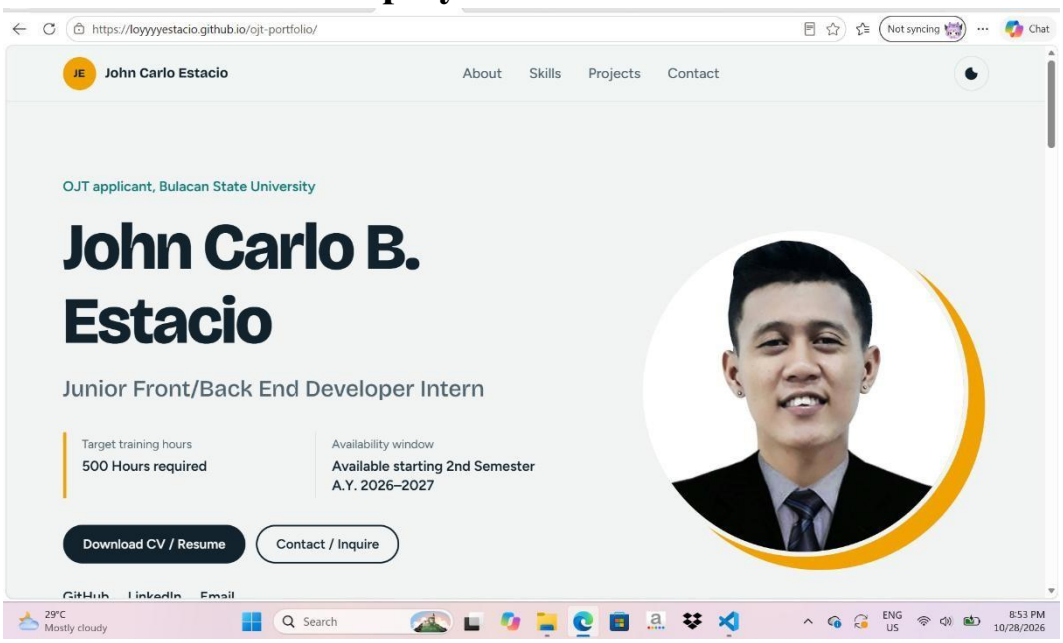
2. OJT Industry Readiness

The **Featured Projects** section best shows I am ready for industry training. Each card states my specific role, shows the actual interface or hardware, and links to the code. The Arduino card includes an interactive preview in which a temperature slider controls how fast a fan spins, which lets a recruiter see how I think about testing and debugging, not just read about it. Together with the Contact section, it gives a recruiter everything needed to evaluate me and reach out.

3. Future Upgrades

Before sending the portfolio to companies, I plan to add a short walkthrough video and a live demo link for LearnHive, so recruiters can see the offline feature working. I would also replace the illustrative temperature thresholds in the fan preview with the real values used by my Arduino controller.

Section 6: Proof of deployment and screenshots



Full page desktop view



Full page mobile view